

Mitigating Catastrophic Forgetting in Task-Agnostic Continual RL via GMM-Routed Replay

Vincenzo Nardelli

1. INTRODUCTION

Continual Reinforcement Learning (CRL) agents are trained on sequential tasks with the objective of maintaining performance without suffering from catastrophic forgetting. Scaling these systems to real-world robotic applications is particularly challenging, as we rarely know when a task changes. In this project, we integrated the ProtoCRL architecture into a Soft Actor-Critic (SAC) framework. To handle task shifts we introduced a novelty-detection mechanism based on negative log-likelihood that dynamically relocates Gaussian Mixture Model (GMM) cluster means to the center of novel state distributions. We evaluated our algorithm on the Continual-World benchmark (based on Meta-World v2), focusing on the first four tasks due to computational constraints.

2. IMPLEMENTATION DETAILS

2.1 Architectural Adaptation

The original ProtoCRL paper evaluated a DDQN architecture where the encoder’s output fed both the Variational GMM (VGMM) and the Q-network. Because Q-values are unstable in continuous SAC implementations, we attached the VGMM only to the actor. This allows the Q-network to serve as a stable anchor, while the actor and VGMM follow its predictions. The encoder acts as the actor’s backbone, routing its output to both the GMM heads (μ , σ , and π) and the remainder of the actor network.

2.2 Decoupling the ELBO Loss

We detached the GMM’s gradient updates from the encoder. We observed that maximizing the ELBO loss:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(z|x)} [\log p(x|z)] - D_{\text{KL}}(q(z|x) \parallel p(z)) \quad (1)$$

placed the GMM in direct opposition to the actor, driving the encoder’s standard deviation to zero. Once the variance collapsed, the Mahalanobis distance between states became near 0, artificially minimizing the ELBO without requiring the GMM to adjust its variance (σ). Even when routing a fraction (e.g., 0.1) of the GMM gradients back to the encoder, the representation still collapsed. As a consequence, we decoupled the gradients and strictly constrained σ near zero to prevent numerical singularities.

2.3 Geometric Constraints (The Hypersphere)

Standard SAC actors typically utilize ReLU (and occasionally Tanh) activations. Our initial implementation used ReLU, but we hypothesize that because of it the network exploited the truncation of negative values, driving activations to zero to minimize the Mahalanobis distance. To resolve this, we replaced the final ReLU layer with an L_2 normalization, projecting the representation onto a hypersphere of radius $R = 20$. This forced all latent states to maintain a constant magnitude, completely eliminating the zero-collapse failure mode while ensuring sufficient spatial separation to prevent cluster overlap.

2.4 Vanishing Gradients and Novelty Routing

Projecting into high-dimensional space ($\mathbb{R}^{256}$) with $R = 20$ introduced a new challenge: vanishing gradients. Because distances scale drastically in high dimensions, the likelihood of new states belonging to distant clusters dropped to nearly zero. The posterior collapsed, allocating all probability to clusters within the initial state region and ignoring the rest. With the posterior at zero, gradients stopped flowing, freezing the unused GMM means.

To bypass this, we added a custom routing mechanism. Assuming $\sigma \approx 0$, the negative log-likelihood becomes proportional to the Mahalanobis distance:

$$D_M(x, \mu) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)} \quad (2)$$

We use this metric to check if a state is novel. If a state cannot be represented by the active GMM clusters, it triggers a task shift. The algorithm selects the cluster with the lowest posterior probability and teleports its mean directly to the mean of the novel states. This reawakens the posterior, allowing gradients to flow and the GMM to resume local boundary refinement.

2.5 Initialization and Buffer Sampling

To prevent initial cluster overlap, means are initialized via a distance-based multinomial distribution, pushing new clusters away from existing ones, followed by a K-means adjustment on warm-up data.

For replay buffer sampling, we dynamically construct batches using a fixed percentage from the default table (table_0), splitting the remainder evenly across all populated historical tables. This ratio balances plasticity and stability: heavily weighting the default table mimics a standard replay buffer (accelerating forgetting), while heavily weighting historical tables starves the network of on-policy data, destroying sample efficiency and causing off-policy divergence. Finally, due to varying reward magnitudes across Meta-World environments, all environment rewards are normalized to stabilize the critic.

3. TESTS AND RESULTS

We evaluated the architecture on 4 ContinualWorld tasks for 400k steps each (though 1M+ steps per task is standard in literature). We tracked evaluation success every 50k steps, final average success, and average forgetting.

3.1 Baseline Comparison

The first comparison is against a standard SAC baseline. To have the same network capacity, the SAC baseline used a standard ReLU activation instead of ProtoCRL’s hypersphere projection. As expected, SAC demonstrates absolute catastrophic forgetting the moment its Replay Buffer is overwritten with new data. Furthermore, SAC’s performance on Task 0 collapsed prematurely at 350k steps (before the task even finished). We hypothesize that ProtoCRL’s L_2 hypersphere projection acts as a strict regularization constraint on activation magnitudes, making the actor sturdier during extended single-task training compared to an unconstrained ReLU layer.

In our best run, ProtoCRL achieved an Average Forgetting of 0.413 while maintaining an Average Success of 0.412 (noting that peak success was limited primarily by the short 400k step training).

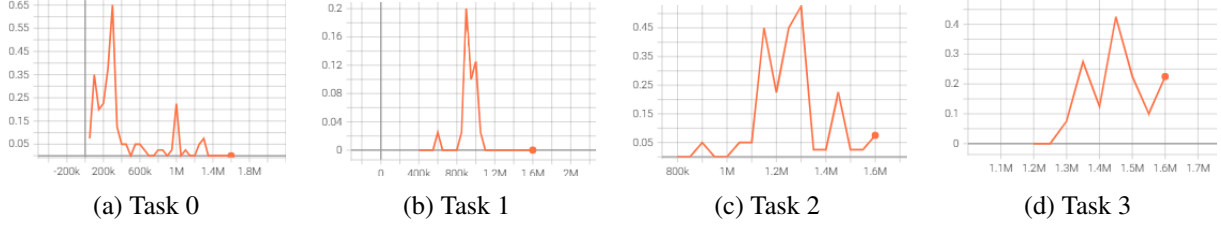


Figure 1: Standard SAC’s running successes across 4 tasks.

3.2 Run-to-run Variance and Cluster Exhaustion

We observed that the algorithm is highly run-dependent. We compared two distinct runs (Run A and Run B) executed with the exact same hyperparameter configuration.

If we compare their performance on Task 0: In Run A, evaluation success peaked at 0.75 around step 650k and then maintained a stable retention floor, averaging 0.27 after the task ended. In Run B, the agent achieved 1.0 success in just 300k steps, and after Task 0 ended, it maintained a highly successful retention average of 0.768. However, at step 1M, Run B’s performance violently collapsed to 0.

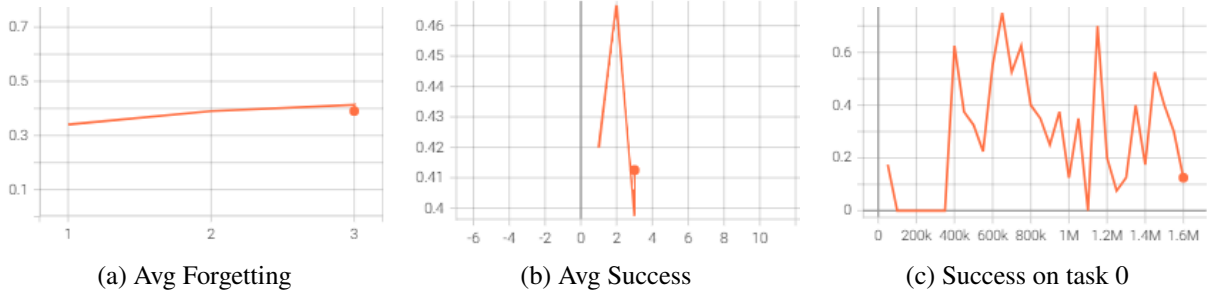


Figure 2: Evaluation of test A

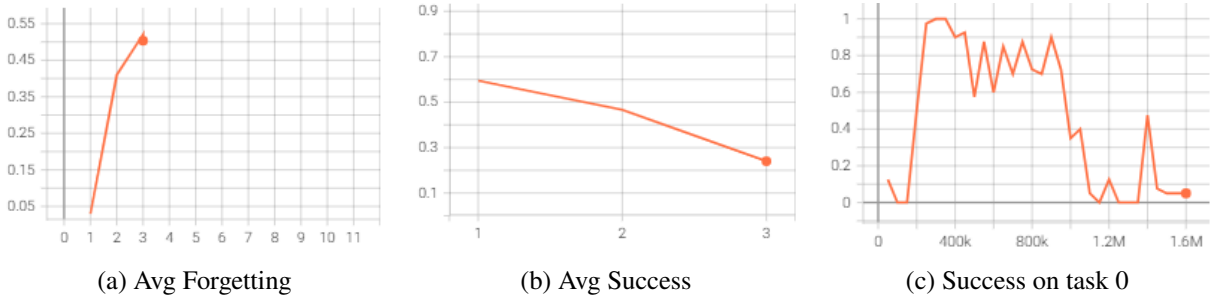
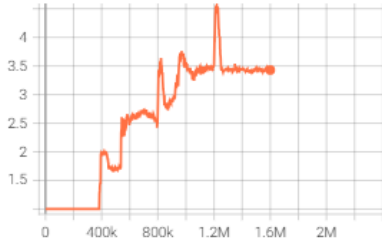
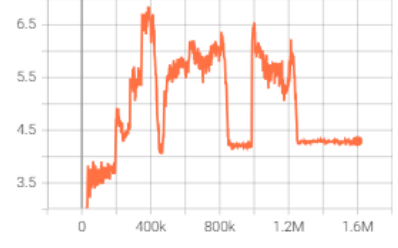


Figure 3: Evaluation of test B

We think that the cause of run B forgetting is cluster exhaustion. In Run A, the Effective_Num_Used metric scaled efficiently, starting from 1 and peaking at 4.55. Conversely, Run B started at 3.7 and wildly oscillated to a peak of 6.7. Because Run B consumed too many clusters on early tasks, it exhausted the GMM’s capacity. When later tasks required new clusters, the system was forced to overwrite early data, triggering the late-stage collapse.

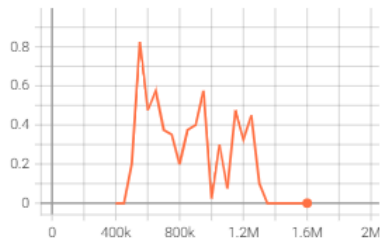


(a) Test A



(b) Test B

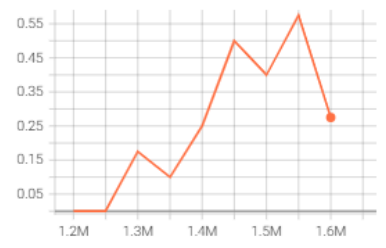
Figure 4: Number of cluster used



(a) Task 1



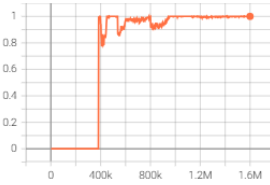
(b) Task 2



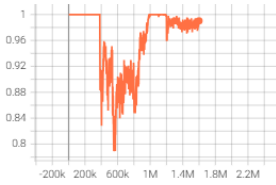
(c) Task 3

Figure 5: Success across tasks in test B

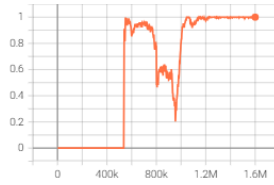
Despite this variance, we can state that the core routing mechanism works. Analyzing the routing stability of both runs shows that clusters remain bound to specific latent state regions, with transitions maintaining highly stable routing averages once they become active.



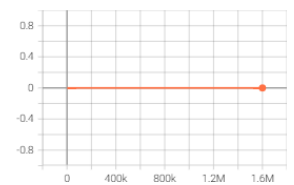
(a) Table1



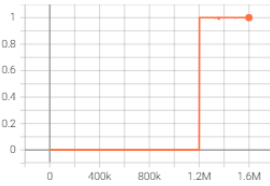
(b) Table2



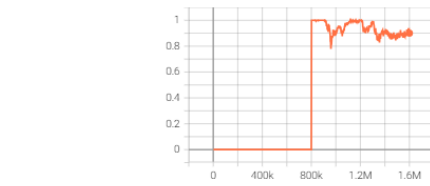
(c) Table3



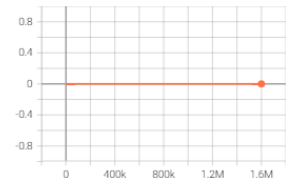
(d) Table4



(e) Table5



(f) Table6



(g) Table7

Figure 6: GMM cluster routing stability over time in test A

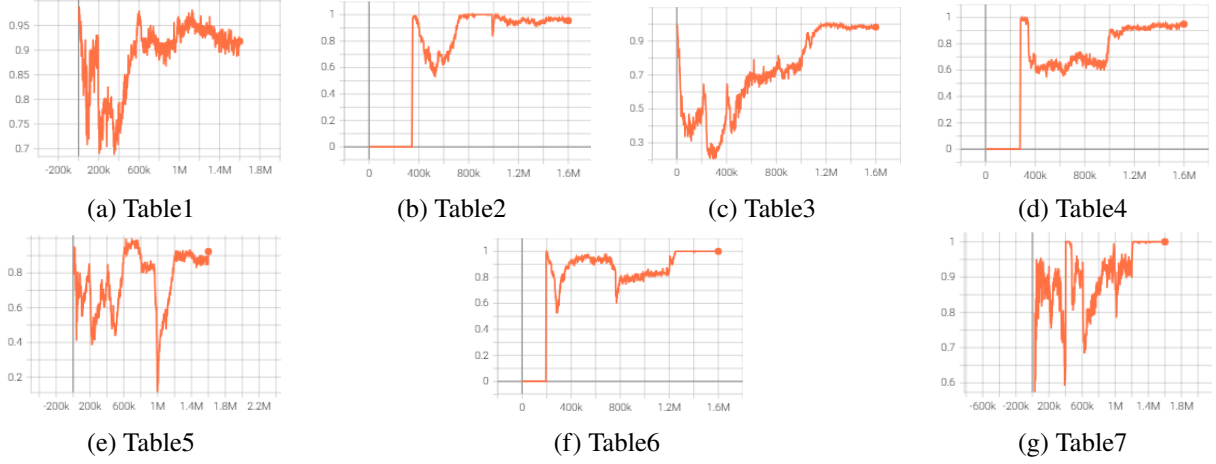


Figure 7: GMM cluster routing stability over time in test B

4. CONCLUSIONS

We successfully implemented ProtoCRL integrated with an off-policy SAC architecture and a novelty-based mechanism. This architecture demonstrates a definitive improvement in mitigating catastrophic forgetting compared to classic SAC. However, the algorithm is heavily run-dependent, requiring multi-run evaluations to fully map its stability bounds, which was outside the scope of our compute budget.

5. LIMITATIONS & FUTURE WORK

- **Fixed Entropy:** To limit the number of unstable variables, we fixed the entropy coefficient ($\alpha = 0.2$) instead of using SAC’s automated entropy tuning. This caused policy oscillation in late stages since exploration could not be scaled down near convergence.
- **Compute Constraints:** We lacked the computational power to run standard 10-run benchmarks per configuration, which is necessary to reduce variance and establish definitive statistical bounds.
- **Buffer Sampling Tuning:** Further tuning is required for the table_0 sampling fraction. Finding the optimal ratio of active-task data to historical data is highly sensitive and directly impacts both plasticity and forgetting.
- **Training Length:** Increasing the step count per task (e.g., to 1M+) would likely stabilize the baseline average success and potentially yield further insights into the long-term limits of the GMM’s retention.